VIDEO GAME DESIGN AND DEVELOPMENT DEGREE

PROMOTION 2024-2025

Final Practice

Project Documentation

Author	Xavier Canals Hernaez
Original version	15/01/2025
Updated version	12/10/2026
Target	Windows x64 - OpenGL 3.3

Contents

General Description	3
Project Architecture	3
Terrain and Lighting	4
Rendering Components	5
Build, Window and Camera	6
Terrain Rendering and Textures	7
Transparency, Lighting and Skybox	8
Limitations and Conclusion	9

This revision documents the corrected terrain topology, height-based terrain materials, transparent rendering order, camera-relative skybox, restored 1024 x 576 output, and per-fragment Blinn-Phong lighting.

General Description

Final Practice is a modular C++ and OpenGL 3.3 application that renders an interactive 3D scene. SDL2 creates the window and OpenGL context, GLAD loads OpenGL functions, SOIL2 loads raster textures, Assimp imports FBX meshes, and GLM handles matrices and vectors.

The scene contains a table, chairs, beer mugs, a transparent fishbowl, an animated transparent crystal, a height-map terrain, and a cubemap skybox. The camera can move freely while the skybox remains centered on it.

Project Architecture

Class Window

Creates the SDL window and OpenGL 3.3 context, enables double buffering, selects the requested depth and stencil buffer sizes, configures vertical synchronization, and swaps the front and back buffers.

Class Texture

Loads 2D albedo maps, monochrome height maps, and six-face cubemaps. Albedo textures use repeating coordinates for tiled materials; height maps and cubemaps clamp at their borders. Mipmaps and linear filtering are generated for 2D textures.

Class Shader

Compiles vertex and fragment shaders, links the resulting OpenGL program, reports compilation or linkage errors, and activates the shader before uniforms and draw calls are submitted.

Area	Current state
Graphics API	OpenGL 3.3
Platform	Windows x64 with Visual Studio 2022
Window	Fixed 1024 x 576, centered
Camera input	Mouse look and WASD movement

Terrain and Lighting

Class Terrain

Builds a regular indexed grid with (xSlices + 1) by (zSlices + 1) vertices. Each cell is rendered as two consistently wound triangles. For the configured 100 x 100 subdivisions, this produces 10,201 vertices, 20,000 triangles, and 60,000 valid indices.

The vertex shader samples the monochrome height map to displace vertices. It also samples neighboring heights to derive smooth surface normals. The fragment shader blends grass and rock albedo textures according to normalized height, then applies the same Blinn-Phong lighting used by imported meshes.

Class Lighting

Stores a white omnidirectional point light at world position (3, 5, 3). The light is transformed to view space every frame so it remains fixed in the scene while the camera moves.

The implemented local illumination model contains ambient, Lambert diffuse, and Blinn-Phong specular terms. Every direction is normalized and the inverse-transpose normal matrix keeps normals correct after model transforms.

```
H = normalize(L + V)

color = ambient + attenuation * (diffuse + specular)
```

Area	Current state
Ambient intensity	0.18
Diffuse intensity	0.85
Specular intensity	1.00
Attenuation	1 / (1 + 0.014 d + 0.0007 d^2)
Shadows	Not implemented; Blinn-Phong is a local reflection model

Rendering Components

Class MeshLoader

Imports the first mesh in an FBX or compatible Assimp scene, creates coordinate, UV, normal, and index buffers, and renders it with model, view, projection, and normal matrices. Textured and untextured objects share the same per-fragment Blinn-Phong model.

Opaque materials use moderate specular response and a shininess exponent of 32. Transparent materials use a stronger response and an exponent of 96 to produce tighter highlights. The texture color acts as material albedo.

Class Skybox

Loads six PNG images into a cubemap. Camera translation is removed from the view matrix, which keeps the cube centered at the camera regardless of movement while preserving camera rotation. The vertex shader places it at the far plane and the render pass uses GL_LEQUAL without writing depth.

Class Scene

Owns the camera and every scene object, processes movement and crystal animation, and enforces the required render order: opaque meshes, terrain, skybox, crystal, and finally fishbowl.

Class Postprocess

Contains framebuffer and full-screen rendering support intended for post-processing. The effect remains disabled in Scene because this subsystem is not yet integrated reliably.

Area	Current state
Imported formats	Formats supported by Assimp, with FBX used by this scene
Mesh indices	Unsigned 16-bit indices
Transparency	Source alpha blending with depth writes disabled
Post-processing	Present in code but disabled

Build, Window and Camera

Build configuration

The Visual Studio 2022 solution supports Debug and Release configurations. The maintained target is x64. Static libraries are resolved from the project libraries directory and include SDL2, GLAD, SOIL2, Assimp, and zlib.

- Open Final Practice.sln in Visual Studio 2022.
- Select x64 and either Debug or Release.
- Build the solution, then run with the project directory as the working directory so relative asset paths resolve.
- Release output is copied to binaries/executable/Final Practice.exe for distribution.

Window and projection

The application opens a centered fixed-size window at the original 1024 x 576 dimensions. Scene initialization propagates the same dimensions to the camera ratio, render viewport, terrain, meshes, and framebuffer-related objects. The aspect ratio remains 16:9.

Camera

The camera starts at world position (0, 0, -2.5), preserving the original scene spawn. WASD moves along the forward and right vectors, while dragging with the left mouse button changes yaw and pitch. Vertical rotation is clamped to avoid flipping.

The skybox is centered at the camera only in view space. This does not modify the camera world position or movement and therefore does not change the spawn.

Area	Current state
Debug executable	project/visual-studio-2022/x64/Debug
Release executable	project/visual-studio-2022/x64/Release
Distributed executable	binaries/executable

Terrain Rendering and Textures

Corrected topology

The earlier implementation allocated xSlices by zSlices vertices but generated indices as if an extra boundary row and column existed. It also changed the vertex traversal direction between rows. Both issues could produce invalid indices or a terrain that appeared as disconnected lines.

The corrected implementation allocates (xSlices + 1) by (zSlices + 1) vertices, traverses every row consistently, and derives triangle indices from the actual number of vertices per row. Rendering now uses GL_TRIANGLES.

Height-based material

The original height map still controls displacement. Two generated albedo maps provide grass and weathered rock. Texture coordinates repeat twelve times across the terrain to avoid stretching and preserve surface detail.

- Below 42 percent normalized height: grass.
- Between 42 and 68 percent: smooth grass-to-rock transition.
- Above 68 percent: rock.

Terrain normals

Neighboring height samples define tangent vectors along the X and Z axes. Their cross product produces an object-space normal, which is transformed with the same inverse-transpose matrix used by imported meshes. This lets slopes respond naturally to the point light.

Area	Current state
Height map	height_map.png, monochrome
Low material	terrain_grass_albedo.png
High material	terrain_rock_albedo.png
Maximum local height	5 units before model scaling

Transparency, Lighting and Skybox

Transparency

The fragment shader now writes transparency to the alpha channel instead of multiplying the complete RGBA vector. Transparent meshes enable source-alpha blending and disable depth writes while keeping depth testing active.

Render order is essential. Opaque geometry fills the depth buffer first. The animated crystal is then rendered before the fishbowl because it is enclosed by the bowl. This prevents the transparent shell from incorrectly hiding the crystal.

Blinn-Phong lighting

Lighting is calculated per fragment rather than only per vertex. The diffuse term uses max(dot(N, L), 0). The specular term uses pow(max(dot(N, H), 0), shininess), where H is the normalized halfway vector between the light and view directions.

This model controls how visible surfaces react to light. It does not determine whether another object blocks the light, so projected shadows are intentionally outside the current implementation.

Camera-relative skybox

Removing translation from the view matrix makes the skybox follow camera movement without appearing to move closer. Assigning clip_position.xyww places every skybox fragment at maximum depth. GL_LEQUAL lets the background fill only pixels not occupied by nearer geometry.

Area	Current state
Opaque pass	Meshes and terrain
Background pass	Skybox at the far plane
Transparent pass	Crystal followed by fishbowl
Projected shadows	Not implemented

Limitations and Conclusion

Current limitations

- No shadow mapping, reflection probes, global illumination, or physically based rendering.
- Post-processing code exists but is disabled in the active scene.
- MeshLoader renders the first mesh from an imported scene and supports one albedo texture per object.
- The window is fixed at 1024 x 576 and does not currently process resize events.
- Transparent objects are manually ordered; a larger scene would require distance sorting or another transparency technique.

Conclusion

The revised project provides a consistent raster-rendering pipeline for opaque, transparent, textured, and height-mapped geometry. Terrain indices are valid, the skybox behaves as an infinite background, transparent objects preserve scene depth, and all visible materials use a coherent per-fragment Blinn-Phong model.

The implementation remains deliberately aligned with the course material: ambient, Lambert diffuse, and Blinn-Phong specular reflection are demonstrated without presenting shadow projection as part of the local lighting model.

Area	Current state
Terrain	Indexed triangles, height displacement, blended grass and rock
Lighting	Per-fragment Blinn-Phong point light
Transparency	Correct alpha output and explicit render order
Skybox	Camera-relative cubemap rendered at far depth
Output	Updated Windows x64 Release executable